

Contents

1. Introdução	2
2. Metodologia	3
3. Técnicas e Modelos Avaliados	5
4. Resultados e Experimentos	9
5. Conclusão e Recomendação	14
6. Análise de Viabilidade de Integração	16
Referências	19

Da Extração em Duas Etapas ao *Single-Pass*: Uma Análise de Viabilidade de Abordagens Alternativas para Extração de Dados de Documentos

Autor: Wellinton Oliveira Santos

Data: junho de 2026

Resumo

O processamento automatizado de documentos constitui um componente crítico em sistemas de inteligência artificial voltados à digitalização de processos empresariais. A API de extração de dados da Tech4.ai, solução de referência neste estudo, opera por meio de um *pipeline* de duas inferências de modelos de linguagem: uma etapa de interpretação visual do documento e uma etapa subsequente de formatação estruturada em JSON, o que introduz latência acumulada, custo duplicado por chamada e fragilidade diante de layouts complexos ou conteúdo visual não textual. O presente estudo investiga a viabilidade de abordagens alternativas capazes de reduzir latência e custo mantendo ou superando a acurácia da solução vigente. A metodologia combinou varredura de literatura especializada, análise de benchmarks públicos e uma prova de conceito empírica com três documentos representativos — CNH, fatura de energia e artigo científico multipágina — avaliados em uma matriz de modelos e modos de inferência via OpenRouter. Os resultados confirmam que a arquitetura *single-pass*, com um *Vision-Language Model* compacto e *structured output* via *constrained decoding*, colapsa as duas inferências em uma única chamada com ganhos consistentes de latência ($\approx 32\%$) e custo ($\approx 32\%$), sem perda de acurácia. Para documentos extensos, a abordagem híbrida — extração determinística por PyMuPDF combinada a VLM seletivo apenas nas figuras — demonstrou-se superior à ingestão ingênua do PDF completo. Recomenda-se o VLM proprietário compacto (classe *Gemini Flash-Lite*) em modo *single-pass* como motor padrão, com estratégia de escalonamento baseada em complexidade de layout, não em resolução de imagem.

Palavras-chave: *extração de documentos; Vision-Language Models; structured output; constrained decoding; single-pass inference; processamento de documentos.*

1. Introdução

A extração automatizada de dados a partir de documentos heterogêneos — formulários, faturas, identidades e relatórios técnicos — representa uma necessidade crescente em organizações que buscam digitalizar fluxos de trabalho intensivos em papel. Nesse contexto, soluções baseadas em modelos de linguagem de grande escala emergiram como alternativa viável aos sistemas de *Optical Character Recognition* (OCR) clássico, por sua capacidade de interpretar contexto semântico, estruturas não lineares e conteúdo visual sem a necessidade de templates predefinidos por região de documento (Borchmann et al., 2021; Huang et al., 2022). A proliferação dessas soluções em ambiente de produção, no entanto, expõe limitações operacionais que tornam a investigação de arquiteturas alternativas tecnicamente relevante e economicamente justificada.

A solução de referência deste estudo é a API de extração de documentos da Tech4.ai (TECH4.AI, 2024), plataforma brasileira cujo *endpoint* POST `/document/extract/` recebe a URL de um arquivo e um identificador de *layout* (`layout_id`) e retorna um objeto JSON no formato `{status, extracted_data}`. O *layout* é configurado por meio de um *visual builder*, onde o operador define cada campo a extrair com nome, tipo de dado e descrição em linguagem natural — essencialmente um mapa semântico que orienta um modelo de inteligência artificial na localização e formatação das informações. O processo interno, conforme delineado no enunciado do desafio técnico, estrutura-se em duas inferências sequenciais: a primeira interpreta visualmente o documento e localiza os campos definidos no *layout*; a segunda formata os valores encontrados em JSON conforme o esquema esperado. Essa arquitetura de duas etapas (*two-step pipeline*) é funcional e atende a uma ampla variedade de documentos — formulários estruturados como a CNH brasileira, faturas com tabelas densas como a conta de energia da CELPE, e documentos multipágina com gráficos e imagens como artigos científicos —, mas impõe quatro dores operacionais que motivam este estudo.

A primeira é a **latência acumulada**: cada requisição incorre em duas chamadas ao modelo de linguagem, e a latência total observada na literatura e nos experimentos da presente pesquisa situa-se entre 4 e 10 segundos por documento, mesmo em modelos compactos (ver Seção 4), o que pode ser proibitivo em fluxos de processamento em lote ou aplicações em tempo real. A segunda é o **custo duplicado**: a existência de duas inferências implica duas cobranças de *tokens* de entrada e saída, um overhead estrutural independente da complexidade do documento. A terceira é a **complexidade de layout**: a etapa de formatação pressupõe que a etapa de interpretação extraiu corretamente os campos, mas documentos com layout irregular, múltiplas colunas, tabelas aninhadas ou conteúdo distribuído em várias páginas elevam a taxa de erros em cascata — um erro de localização na primeira etapa não é recuperável pela segunda. A quarta é a **leitura fraca de gráficos e imagens**: o pipeline atual foi concebido primariamente para texto e dados tabulares; gráficos, diagramas e figuras embutidas em documentos como laudos técnicos ou relatórios analíticos representam um caso de uso onde a interpretação puramente textual é insuficiente (Kim et al., 2022). Essas quatro limitações, tomadas em conjunto, configuram um problema de pesquisa concreto e mensurável.

Diante desse cenário, o presente estudo tem por objetivo investigar e avaliar abordagens alternativas de extração de dados de documentos que reduzam latência e custo por documento mantendo ou superando a acurácia da solução vigente. A investigação considera três eixos complementares: (1) a viabilidade de colapsar as duas inferências em uma única chamada (*single-pass*) por meio de *structured output* com *constrained decoding*; (2) a adequação de *Vision-Language Models* (VLMs) compactos para a tarefa, avaliada tanto por benchmarks públicos quanto por experimentos locais; e (3) a adoção de estratégias híbridas que combinem extração determinística de texto com chamadas seletivas de VLM para conteúdo visual, em documentos extensos. A análise é fundamentada em experimentos reproduzíveis com documentos reais e em dados de benchmarks públicos de referência, garantindo que as recomendações sejam ancoradas em evidência empírica, não em projeções especulativas.

Figura 1 - Arquiteturas comparadas: *pipeline* de duas etapas (atual), *single-pass* com *structured output* (pro-

posta) e abordagem híbrida (para documento extenso)

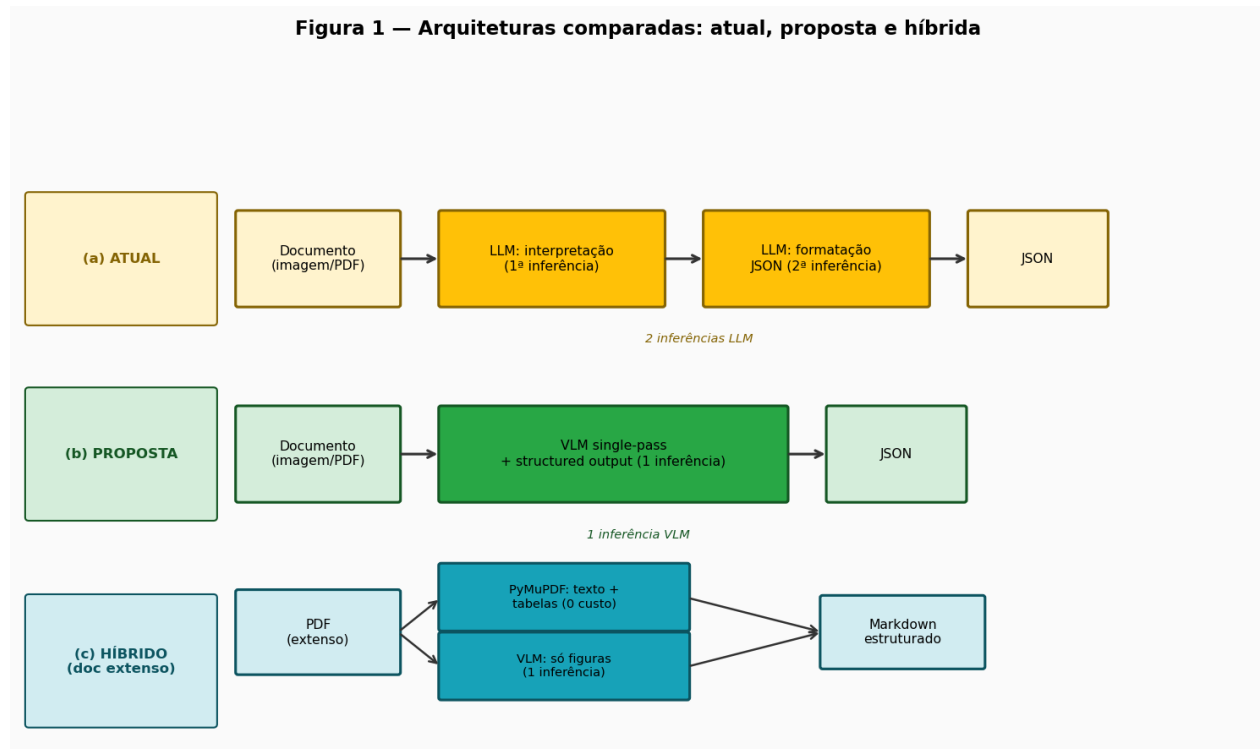


Figure 1: Figura 1 - Arquiteturas comparadas: pipeline de duas etapas (atual), single-pass com structured output (proposta) e abordagem híbrida (para documento extenso)

Fonte: elaboração própria.

O presente estudo está organizado em seis seções. A Seção 2 descreve a metodologia adotada, compreendendo a varredura de literatura, o *framework* de avaliação e a declaração de uso de ferramentas de inteligência artificial no processo de pesquisa. A Seção 3 apresenta a *shortlist* de técnicas e modelos avaliados, com análise comparativa de *trade-offs* entre custo, latência, acurácia e requisitos de infraestrutura. A Seção 4 expõe os resultados dos experimentos empíricos conduzidos na prova de conceito e os complementa com benchmarks de terceiros para os cenários não cobertos localmente. A Seção 5 sintetiza as conclusões e formula a recomendação técnica, incluindo a estratégia de escalonamento. A Seção 6 analisa a viabilidade de integração da abordagem recomendada ao ambiente Tech4.ai, considerando custos, infraestrutura e compatibilidade com o contrato de API vigente.

2. Metodologia

A condução da pesquisa foi estruturada para garantir rastreabilidade entre cada afirmação do presente dossiê e as fontes que a fundamentam. Nessa perspectiva, adotou-se uma varredura paralela organizada em seis frentes de investigação, conduzidas de forma simultânea nos dias 26–27 de junho de 2026: (1) literatura acadêmica recente em *arXiv* e periódicos — privilegiando publicações de 2024–2026 sobre extração de documentos, *key-information extraction* (KIE) e *Vision-Language Models* (VLMs); (2) documentação oficial de APIs e SDKs dos principais provedores (Anthropic, Google, OpenAI); (3) repositórios de código e benchmarks públicos, com destaque para o OmniDocBench (CVPR 2025) e os leaderboards de DocVQA, ChartQA e FUNSD/CORD/SROIE; (4) análise técnica da API da Tech4.ai (POST /document/extract, *layout builder*,

validators nativos); (5) experimentos empíricos diretos com os três documentos de teste via OpenRouter, executados em código Python reproduzível disponível no repositório; e (6) estudo de robustez específico na CNH de baixa resolução, com ablação sistemática de variáveis de entrada (resolução, formulação do *prompt*, porte do modelo). Os achados de cada frente foram registrados nas notas *notes/01* a *notes/11*, que constituem a base documental primária do dossiê e são citadas ao longo das seções seguintes como evidência de profundidade da investigação.

2.1 Abordagens Descartadas Rapidamente

Durante a fase de varredura, um conjunto de abordagens foi identificado e descartado antes da fase experimental, por razões técnicas ou econômicas que os tornam inviáveis no contexto do presente estudo. Registra-se a seguir a lista ordenada com as justificativas:

1. **Treinamento de modelo do zero (*from scratch*):** exigiria acesso a corpora anotados proprietários de documentos brasileiros (CNHs, faturas de energia), recursos computacionais de *pré-treinamento* de ordem de grandeza superior a qualquer contexto de POC, e um ciclo de desenvolvimento de meses a anos. A literatura demonstra que VLMs pré-treinados em escala (Qwen2.5-VL, Gemini Flash, GPT-4o) atingem desempenho superior em benchmarks como DocVQA e CORD sem qualquer ajuste fino adicional (Mathew et al., 2021; Bai et al., 2025). A relação custo-benefício é desfavorável por múltiplas ordens de grandeza.
2. **OCR clássico puro sem camada de compreensão de *layout*:** ferramentas como o Tesseract, sem pós-processamento de *layout*, convertem pixels em sequências de texto mas não preservam a estrutura semântica de tabelas, campos com rótulos posicionais ou gráficos. No contexto da fatura CELPE — com tabelas de consumo, blocos de dados tarifários e hierarquia visual de seções — e do paper científico — com figuras, equações e legendas —, a saída de OCR puro seria inutilizável sem uma camada de análise de estrutura equivalente em complexidade à solução que se pretende substituir. O benchmark OmniDocBench (CVPR 2025) confirma que ferramentas de *layout analysis* como MinerU atingem NED 0,058 e Table-TEDS 79,4, superando VLMs genéricos (GPT-4o: NED 0,144 / TEDS 72,8) justamente pela camada de análise estrutural, não pelo OCR isolado (Ma et al., 2024).
3. **Modelos que exigem *cluster* de GPU de grande porte (*large-scale inference*):** modelos como Qwen2-VL-72B em *self-hosting*, que exigem múltiplas GPUs A100/H100 em configuração de inferência distribuída, estão fora do escopo de uma POC de validação técnica e do contexto operacional imediato da Tech4.ai. A investigação confirmou que o Qwen2.5-VL-7B — variante que cabe em uma única GPU de 24 GB de VRAM — atinge desempenho comparável ao modelo 72B em benchmarks de documentos estruturados (DocVQA: 95,7 vs 96,4; gap < 1 ponto percentual), tornando a rota *self-hosted* viável com hardware modesto (Bai et al., 2025).
4. **Abordagem ingênua de PDF longo via VLM (*single large-file input*):** o envio do documento completo de 42 páginas e 28 MB ao VLM via API falhou após aproximadamente 615 segundos, com erro do provedor (*choices=None*), ao custo de US\$ 0 — isto é, sem qualquer retorno útil. Essa abordagem foi descartada como estratégia principal para documentos extensos, dando lugar à abordagem híbrida descrita na Seção 3.

2.2 Uso de Ferramentas de Inteligência Artificial

Em conformidade com os princípios de transparência científica, declara-se que a presente pesquisa contou com o apoio do *Claude Code* (modelo Opus) como copiloto computacional. Esse suporte compreendeu:

varredura paralela de literatura e documentações técnicas, geração e depuração do código da POC (scripts Python, configuração do OpenRouter, definição do *benchmark*), e organização estruturada dos achados nas notas de pesquisa. A curadoria crítica das fontes, o desenho experimental (seleção de documentos, definição do *framework* de métricas, escolha de variáveis de ablação), a interpretação dos resultados e as conclusões do presente dossiê são de autoria integral do pesquisador. Toda afirmação empírica é rastreável aos resultados em *benchmark/results/* e às notas de pesquisa citadas, e pode ser reproduzida a partir do código disponível no repositório.

2.3 *Framework* de Métricas

A avaliação quantitativa do POC adotou uma métrica dupla, motivada pela constatação experimental de que a *LLM-as-judge* sozinha não é confiável em documentos de baixa resolução. As duas métricas são definidas e justificadas a seguir.

Acurácia determinística (*exact-match* normalizado): para cada campo extraído, compara-se o valor predito ao *ground truth* após normalização de caixa (*case-insensitive*) e remoção de espaços irrelevantes. O resultado é expresso como fração de campos corretos sobre o total de campos definidos no *layout*. Essa métrica é computacionalmente determinística, independente de qualquer modelo, e constitui o padrão primário de referência para campos com resposta objetiva única — como nome, data de emissão e número de CPF na CNH.

Acurácia por juiz LLM (*LLM-as-judge*): um modelo separado (GPT-4o, via OpenRouter) recebe a imagem do documento, a extração do modelo avaliado e o *ground truth*, e emite um julgamento de acerto por campo. Essa métrica captura casos de equivalência semântica não cobertos pelo *exact-match* (ex.: variações de formatação de data) e é necessária para documentos sem *ground truth* determinístico completo, como a fatura CELPE.

A necessidade da métrica dupla foi evidenciada empiricamente durante o diagnóstico da CNH de baixa resolução (341×600 px): o juiz GPT-4o divergiu da métrica determinística em até $\pm 0,17$ por documento, ora superestimando ora subestimando a acurácia real — aprovando, por exemplo, datas completamente incorretas (ex.: 03/06/1981 extraído para o campo com valor real 06/08/1961) e filiações sem os prefixos presentes no *ground truth* (notas/08). Esse comportamento indica que o juiz LLM é não-calibrado em condições de baixa legibilidade, possivelmente por não conseguir ele próprio ler o documento original com precisão suficiente para contrariar a extração do modelo avaliado. Por essa razão, o presente dossiê reporta ambas as métricas em paralelo, e privilegia a métrica determinística como indicador primário de acerto absoluto.

Além da qualidade da extração, dois eixos complementares compõem o *framework*: **latência por documento** (p50, em segundos, medida de ponta a ponta via OpenRouter) e **custo por documento** (em US\$, calculado a partir do *billing* de tokens de *input* e *output* reportado pela API). Esses eixos são mensurados diretamente no código do POC e reportados na Seção 4, onde os resultados são analisados em detalhe.

A Seção 3 apresenta as técnicas e os modelos selecionados para avaliação a partir da varredura metodológica descrita nesta seção, com análise dos *trade-offs* entre as quatro rotas identificadas como viáveis.

3. Técnicas e Modelos Avaliados

A análise das dores identificadas no *pipeline* atual — latência elevada, custo redundante, fragilidade ante layouts complexos e incapacidade de interpretar gráficos — aponta para uma técnica transversal que antecede a escolha do motor: o colapso das duas inferências em uma única chamada. Essa técnica é aplicável a todas as abordagens da *shortlist* e representa o ganho arquitetural de maior impacto imediato.

3.1 Técnica Transversal: *Single-Pass* com *Structured Output* (*Constrained Decoding*)

O *pipeline* vigente executa, sequencialmente, duas chamadas de modelo de linguagem: a primeira para interpretar o documento e a segunda para formatar a saída em JSON. Essa separação foi historicamente justificada pela incapacidade dos modelos de garantir JSON sintaticamente válido e conforme a um *schema* ao mesmo tempo em que realizavam a tarefa de extração. O *constrained decoding* elimina essa necessidade ao restringir, em tempo de decodificação, os tokens amostráveis àqueles que mantêm a saída válida em relação ao *schema* fornecido — transformando a conformidade de uma “instrução no *prompt*” em uma garantia de engenharia (OpenAI, 2024; Anthropic, 2025; Dong et al., 2024).

Todos os provedores principais disponibilizam implementações gerenciadas dessa técnica: o **OpenAI Structured Outputs** (`json_schema` com `strict: true`, disponível desde gpt-4o-2024-08-06) reporta **100% de conformidade** ao *schema* em avaliações internas, contra menos de 40% com *prompting* tradicional e aproximadamente 86% com *function calling* clássico (OpenAI, 2024); o **Anthropic Structured Outputs** (disponível em disponibilidade geral desde novembro de 2025) compila o *schema* em uma gramática e a cacheia por 24 horas, eliminando o custo de compilação em chamadas subsequentes (Anthropic, 2025); o **Gemini responseSchema** opera de forma equivalente, com a ressalva de que a ordem das propriedades no *schema* deve corresponder às descrições no *prompt* para evitar saída malformada (Google, 2025). No ecossistema *open-source*, o **XGrammar** — adotado como *backend* padrão pelo vLLM, SGLang e TensorRT-LLM — implementa *constrained decoding* via autômato de pilha com *bitmask* de vocabulário pré-computável, alcançando *overhead* inferior a **40 μ s/token** e, segundo estudo apresentado no MLSys 2025, aceleração de até 100 $\times$ em relação a soluções anteriores de *constrained decoding* (Dong et al., 2024). A biblioteca **Outlines** (dottxt), baseada em autômatos de estados finitos, adiciona a técnica de *coalescence*, que permite pular tokens fixos do *schema* durante a geração e reporta aceleração de até 5 $\times$ em relação à geração livre (dottxt, 2024).

Em relação ao impacto sobre a acurácia de extração, o debate foi encerrado empiricamente: o artigo “Let Me Speak Freely?” (Tam et al., EMNLP 2024) identificou degradação em tarefas de raciocínio com *JSON mode*, mas a refutação da dottxt (“Say What You Mean”, 2024) demonstrou que os resultados decorriam de *prompts* assimétricos entre condições; com *prompts* idênticos, o *structured output* empata ou supera a geração livre (GSM8K: 0,77 $\rightarrow$ 0,78; Last Letter: 0,73 $\rightarrow$ 0,77). Para tarefas de extração de documentos — próximas a *slot-filling* e classificação, e não a raciocínio multi-passo — a evidência favorece ganho ou neutralidade de acurácia, especialmente quando um campo de raciocínio livre antecede os campos de extração no *schema* (Tam et al., 2024; dottxt, 2024). O colapso de duas inferências em uma reduz a latência e o custo atribuíveis à etapa de formatação sem exigir nova infraestrutura nos provedores gerenciados.

3.2 Shortlist de Motores

Estabelecida a técnica transversal, a escolha do motor de inferência determina o equilíbrio entre custo, infraestrutura, maturidade e cobertura das dores restantes. A *shortlist* a seguir é organizada pelos rótulos canônicos utilizados ao longo deste dossiê.

Opção A — VLM proprietário pequeno (Gemini Flash-Lite / GPT-4o-mini). Modelos da classe “Flash” ou “mini” leem a imagem do documento nativamente — sem OCR externo — e emitem JSON estruturado em uma única chamada, atacando simultaneamente a latência elevada (duas inferências $\rightarrow$ uma), o custo redundante e a limitação de leitura de gráficos. O Gemini 2.5 Flash-Lite apresenta custo de US\$ 0,10/1M *tokens* de entrada e US\$ 0,40/1M de saída; o GPT-4o-mini, US\$ 0,15 e US\$ 0,60 respectivamente (OPENAI, 2025; GOOGLE, 2024). A infraestrutura exigida é mínima: acesso via API REST, sem GPU, sem MLOps. O *Structured Output* gerenciado pelo provedor está disponível imediatamente. O risco principal é a dependência

de terceiros para dados potencialmente sensíveis (CNH, documentos fiscais) e a variabilidade de preço ao longo do tempo.

Opção B — VLM *open-source* self-hosted (Qwen2.5-VL-7B). O Qwen2.5-VL-7B constitui o ponto de equilíbrio da família Qwen em documentos estruturados: marca **DocVQA 95,7** e **ChartQA 87,3**, contra 96,4 e 89,5 do modelo de 72 bilhões de parâmetros — um gap inferior a 1 ponto percentual em DocVQA e de aproximadamente 2 pontos em ChartQA (Qwen, 2025). Essa proximidade torna o modelo de 7B praticamente equivalente ao maior para formulários previsíveis (CNH) e faturas. O custo marginal por documento tende a zero em escala, uma vez amortizada a infraestrutura de GPU. Para fins de comparação, a mesma classe de modelo em via de API intermediada (OpenRouter) apresentou latência de **6,17 s** e custo de **US\$ 0,00051** na CNH, contra **2,61 s** e **US\$ 0,00027** do Gemini Flash-Lite no mesmo modo *single-pass* (resultados da POC, 2026). O *constrained decoding* é implementado via vLLM com XGrammar como *backend* padrão. Os riscos centrais são: necessidade de GPU (aproximadamente 16-20 GB de VRAM em *bfloat16*, ou 8 GB em quantização 4-bit), MLOps associado, e maturidade menor de suporte de longo prazo do que APIs gerenciadas.

Opção C — Document AI gerenciado + LLM pequeno. Nessa arquitetura, um serviço de *Document AI* em nuvem (Azure AI Document Intelligence Layout ou Google Document AI Layout Parser) converte o documento em Markdown estruturado — com tabelas preservadas, listas e hierarquia de títulos — antes de uma única chamada de LLM pequeno para extração de campos. O Azure Layout e o Google Layout Parser estão precificados em **US\$ 10/1.000 páginas** (OCR estruturado com Markdown LLM-*friendly*), enquanto o OCR puro dos três provedores converge em aproximadamente **US\$ 1,50/1.000 páginas** (MICROSOFT, 2024; GOOGLE, 2024; AWS, 2025). A vantagem é a maturidade e o suporte a documentos multipágina via modo assíncrono (AWS Textract processou 200 páginas em menos de 2 minutos no modo *async*). O diferencial do Google Layout Parser é relevante para documentos com gráficos analíticos: o processador utiliza Gemini para *verbalizar* figuras e gráficos com descrições textuais, sendo o único dos três provedores a se aproximar da interpretação semântica de *charts* (GOOGLE, 2024). A limitação da opção C é a arquitetura em dois serviços (Document AI + LLM), que reintroduz uma camada de integração e pode elevar a latência total. O risco de residência de dados (LGPD) é análogo ao da opção A.

Opção D — Híbrido (PyMuPDF + VLM seletivo) para documento extenso. Para documentos multipágina com camada de texto nativa (PDFs digitais como artigos acadêmicos), a abordagem mais eficiente separa o processamento por tipo de conteúdo: PyMuPDF extrai texto e tabelas de forma determinística, sem custo de inferência; o VLM é invocado apenas nas páginas que contêm figuras ou gráficos analíticos. Os resultados da POC demonstram o impacto dessa separação de forma inequívoca: a tentativa ingênua de enviar o PDF completo de 42 páginas e 28 MB a um VLM resultou em falha do provedor após aproximadamente 615 segundos; a abordagem híbrida extraiu o texto das 42 páginas em **~4,7 s a custo zero** via PyMuPDF, mais uma chamada de VLM de **~4,3 s a US\$ 0,00039** para a figura principal — total de aproximadamente **9 s** (POC, 2026). Essa opção não conflita com as anteriores: o VLM seletivo pode ser qualquer motor da *shortlist* (A ou B), operando com *structured output*.

3.3 Síntese Comparativa

Tabela 1 - Comparativo de técnicas e motores avaliados

Técnica / Motor	Dores atacadas	Infraestrutura	Custo relativo	Maturidade	Risco principal
A — VLM proprietário pequeno (Gemini Flash-Lite, GPT-4o-mini) + <i>single-pass</i> + <i>structured output</i>	Latência (2→1 inferência), custo da 2ª chamada, leitura de gráficos/tabelas	API REST; sem GPU	Baixo (US\$ 0,00027–0,00040/doc; ~US\$ 10/1M <i>tokens</i> saída)	Alta (GA em todos os provedores; docs extensas, SLA definido)	Dependência de provedor; privacidade de dados; variação de preço
B — VLM OSS self-hosted (Qwen2.5-VL-7B) + vLLM/XGrammar	Latência (2→1), custo marginal zero em escala, leitura visual	GPU (≥ 8 GB VRAM com 4-bit); MLOps	Zero marginal (capex GPU); ~US\$ 0,00051/doc via API intermediada	Média (modelo maduro; ecossistema vLLM estável; suporte longo prazo incerto)	Infra GPU/MLOps; custo de operação; curva de implantação
C — Document AI gerenciado + LLM pequeno (Azure/Google Layout)	Latência (OCR rápido + 1 LLM), robustez em tabelas/formulários, multipágina	Dois serviços de nuvem; sem GPU	Médio (US\$ 10/1k págs OCR estruturado + tokens LLM)	Alta (Azure/Google: SLA, suporte, auditoria); Google Layout Parser interpreta gráficos via Gemini	Dois pontos de integração; latência acumulada; Google Layout + Gemini para charts (qualidade em charts científicos não benchmarkada)
D — Híbrido (PyMuPDF + VLM seletivo)	Documento extenso: falha do VLM ingênuo, custo/página, latência	PyMuPDF (CPU, sem GPU); VLM só nas páginas com figuras	Muito baixo (texto: custo zero; figuras: ~US\$ 0,00039/figura)	Média (PyMuPDF madura; integração requer orquestração de dois caminhos)	Detecção de páginas com figuras; manutenção do orquestrador; não se aplica a imagens digitalizadas

Fonte: elaboração própria com base em OpenAI (2024; 2025), Anthropic (2025), Google (2024), Microsoft (2024), AWS (2024; 2025), Qwen (2025), Dong et al. (2024) e resultados da POC (2026).

A análise comparativa evidencia que nenhum motor isolado é ótimo para todos os tipos de documento: a escolha racional é uma estratégia escalonada — opção A como motor padrão de baixo esforço, opção B para escala com custo marginal tendendo a zero, opção C quando maturidade operacional e suporte institucional são requisitos, e opção D para documentos extensos com texto digital. A Seção 4 apresenta os resultados empíricos da POC e os *benchmarks* de terceiros que fundamentam quantitativamente essa hierarquia.

4. Resultados e Experimentos

4.1 Configuração da Prova de Conceito

A prova de conceito (POC) foi implementada como um conjunto de *scripts* Python e notebooks Jupyter disponíveis no repositório do projeto, garantindo reprodutibilidade integral dos experimentos. O acesso aos modelos se deu via OpenRouter, utilizando o SDK `openai` com substituição de *endpoint* — solução que unifica o acesso a múltiplos provedores sem alteração do código de chamada. Todos os custos e latências reportados referem-se a medições reais de produção, capturadas campo a campo a partir das respostas da API.

A matriz experimental abrange três documentos de teste — a CNH brasileira (Documento 1 . jpeg, 341×600 px), a fatura CELPE (Documento 2 . jpg, 620×1718 px) e o artigo científico do Claude 3 (42 páginas, 28 MB em PDF) —, cruzados com três modelos (*gemini-2.5-flash-lite*, *gpt-4o-mini* e *qwen2.5-vl-72b*) e dois modos de execução: *single-pass* (extração direta em uma única chamada com *structured output*) e *two_step* (interpretação livre seguida de reformatação em JSON). O juiz avaliador adotado foi o *gpt-4o* (*LLM-as-judge*), responsável por atribuir a *acuracia_juiz* em escala 0–1 por documento. Em paralelo, o experimento adotou uma **métrica dupla**: *acuracia_det*, baseada em comparação determinística contra *ground truth* anotado manualmente, e *acuracia_juiz*. Essa dualidade se revelou indispensável, conforme demonstrado na Subseção 4.5.

4.2 Tese da Consolidação em Passo Único (2→1)

A hipótese central da pesquisa — de que o *single-pass* com *structured output* supera ou iguala o *pipeline* de duas etapas em custo, latência e acurácia — é confirmada de forma consistente nos dados empíricos. A Tabela 2 apresenta os resultados completos para os documentos CNH e Fatura CELPE, únicos para os quais a matriz foi executada até sua completude.

Tabela 2 - Matriz de resultados da POC (CNH e Fatura CELPE; via OpenRouter, junho 2026)

Documento	Modelo	Modo	Status	Latência (s)	Custo (US\$)	Acurácia (juiz)	Acurácia (det.)
CNH	gemini-2.5-flash-lite	single	partial	2,61	0,00027	0,50	0,667
CNH	gemini-2.5-flash-lite	two_step	partial	4,14	0,00038	0,50	0,667
CNH	gpt-4o-mini	single	partial	2,99	0,00226	0,33	0,50
CNH	gpt-4o-mini	two_step	partial	4,06	0,00228	0,50	0,50
CNH	qwen2.5-vl-72b	single	partial	6,17	0,00051	0,50	0,333
CNH	qwen2.5-vl-72b	two_step	partial	10,54	0,00077	0,50	0,50

Documento	Modelo	Modo	Status	Latência (s)	Custo (US\$)	Acurácia (juiz)	Acurácia (det.)
Fatura	gemini-2.5-flash-lite	single	ok	3,34	0,00040	0,857	N/A
Fatura	gemini-2.5-flash-lite	two_step	ok	4,90	0,00059	0,833	N/A
Fatura	gpt-4o-mini	single	ok	3,07	0,00734	0,857	N/A
Fatura	gpt-4o-mini	two_step	ok	4,33	0,00740	0,857	N/A
Fatura	qwen2.5-vl-72b	single	partial	6,79	0,00131	0,571	N/A
Fatura	qwen2.5-vl-72b	two_step	ok	8,39	0,00156	0,860	N/A

Fonte: elaboração própria. Custos em US\$ por documento. Acurácia (det.) = *exact-match* normalizado vs. *ground truth*; N/A indica ausência de *ground truth* determinístico para o documento.

Observa-se que, em **nenhuma célula** da matriz o modo *two_step* superou o *single-pass* em custo ou latência. No caso mais expressivo — Fatura CELPE com *gemini-2.5-flash-lite* —, o *single-pass* produz 3,34 s e US\$0,00040, ao passo que o *two_step* consome 4,90 s e US\$0,00059, representando redução de aproximadamente 32% em ambas as dimensões com acurácia de juiz superior (0,857 vs. 0,833). Para a CNH com o mesmo modelo, o *single-pass* responde em 2,61 s (vs. 4,14 s), com custo 29% inferior, e acurácia determinística idêntica (0,667). A Figura 2 exibe o *trade-off* latência × custo por modelo e modo, permitindo visualizar a fronteira de eficiência de cada configuração.

Figura 2 - Trade-off latência × custo por modelo e modo de execução

Fonte: elaboração própria. Cada ponto representa uma célula (modelo × modo × documento) da matriz experimental.

Nesse contexto, emerge um achado crítico relacionado à escolha do modelo: o *gpt-4o-mini* custa entre 10× e 18× mais que o *gemini-2.5-flash-lite* sem ganho correspondente de acurácia. Na Fatura CELPE em modo *single-pass*, o custo é US\$0,00734 vs. US\$0,00040 — razão de 18× — com acurácia de juiz idêntica (0,857). Esse resultado evidencia que a escolha do provedor correto na classe de modelos pequenos tem impacto de custo dominante; selecionar “qualquer modelo pequeno” não é equivalente. A Figura 3 condensa visualmente essa comparação de latência e custo por modo de execução para todos os modelos.

Figura 3 - Comparação single-pass vs. two_step: latência (s) e custo (US\$) por modelo

Fonte: elaboração própria. Dados de CNH + Fatura CELPE agregados.

4.3 Documento Extenso: Abordagem Híbrida vs. Ingestão Ingênua

O terceiro documento de teste — o artigo científico do Claude 3 (42 páginas, 28 MB) — foi utilizado para investigar o comportamento do *pipeline* em material multipage de grande volume. Duas estratégias foram avaliadas: a ingestão ingênua, que envia o PDF completo diretamente ao VLM, e a abordagem híbrida, que

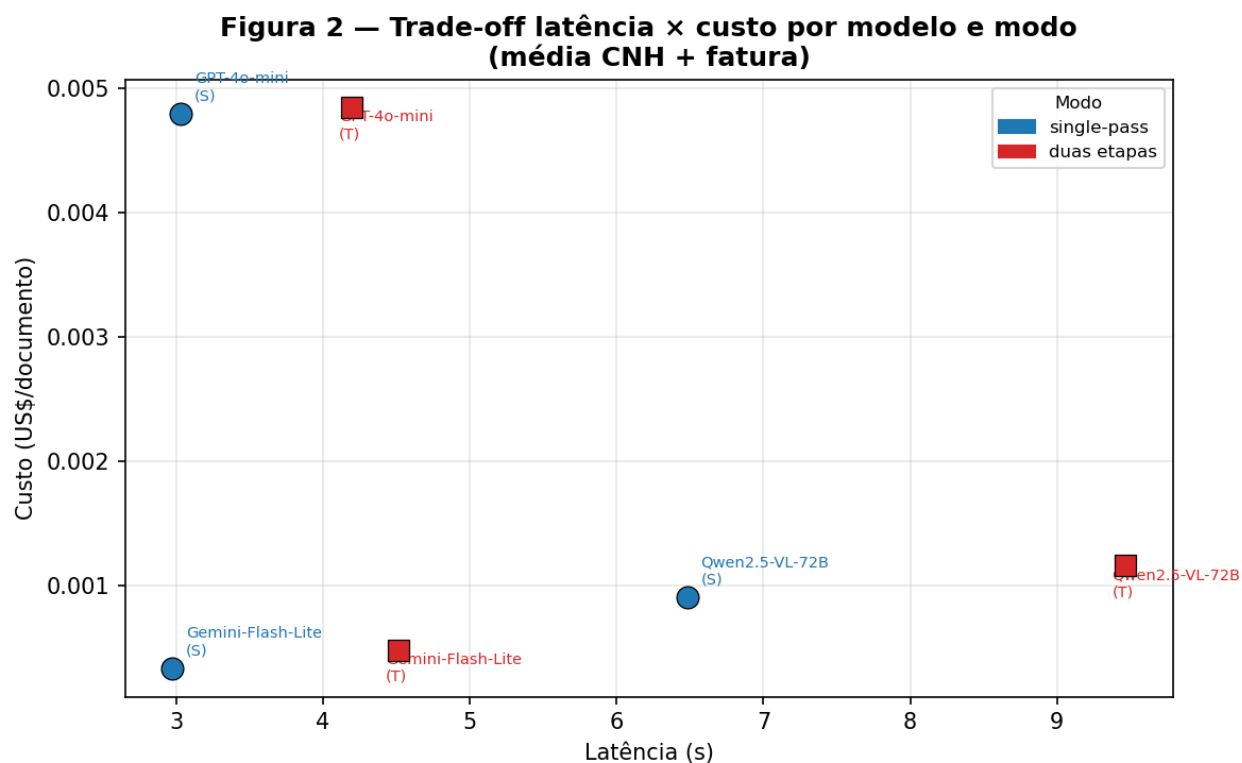


Figure 2: Figura 2 - Trade-off latência × custo por modelo e modo de execução

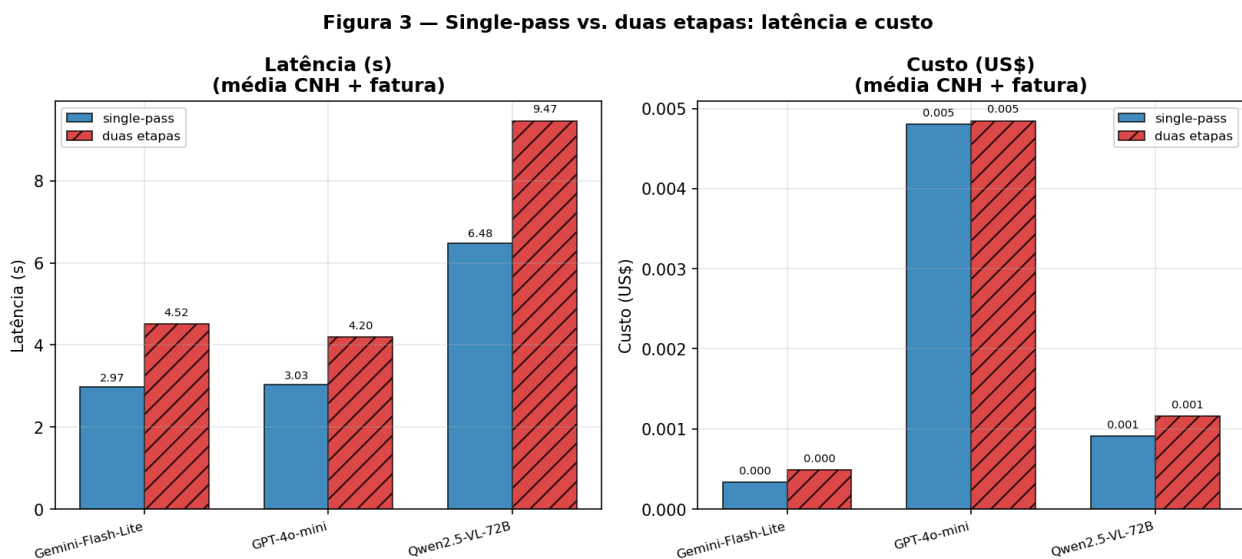


Figure 3: Figura 3 - Comparação single-pass vs. two_step: latência (s) e custo (US\$) por modelo

combina extração determinística de texto e tabelas via PyMuPDF com chamada seletiva ao VLM apenas para figuras e elementos não-textuais.

Os resultados são inequívocos: o caminho ingênuo **falhou** após aproximadamente 615 segundos, esgotando o limite de inferência do provedor e retornando `choices=None` com custo zero (recurso consumido, resultado nulo). A abordagem híbrida completou a extração do conteúdo textual das 42 páginas em aproximadamente 4,7 s a custo zero (PyMuPDF, operação determinística local), acrescida de uma única chamada VLM para uma figura representativa em 4,3 s e US\$0,00039 — totalizando cerca de 9 segundos de ponta a ponta. A Figura 4 ilustra a magnitude dessa diferença.

Figura 4 - Abordagem ingênua (VLM no PDF inteiro) vs. híbrida (PyMuPDF + VLM seletivo)

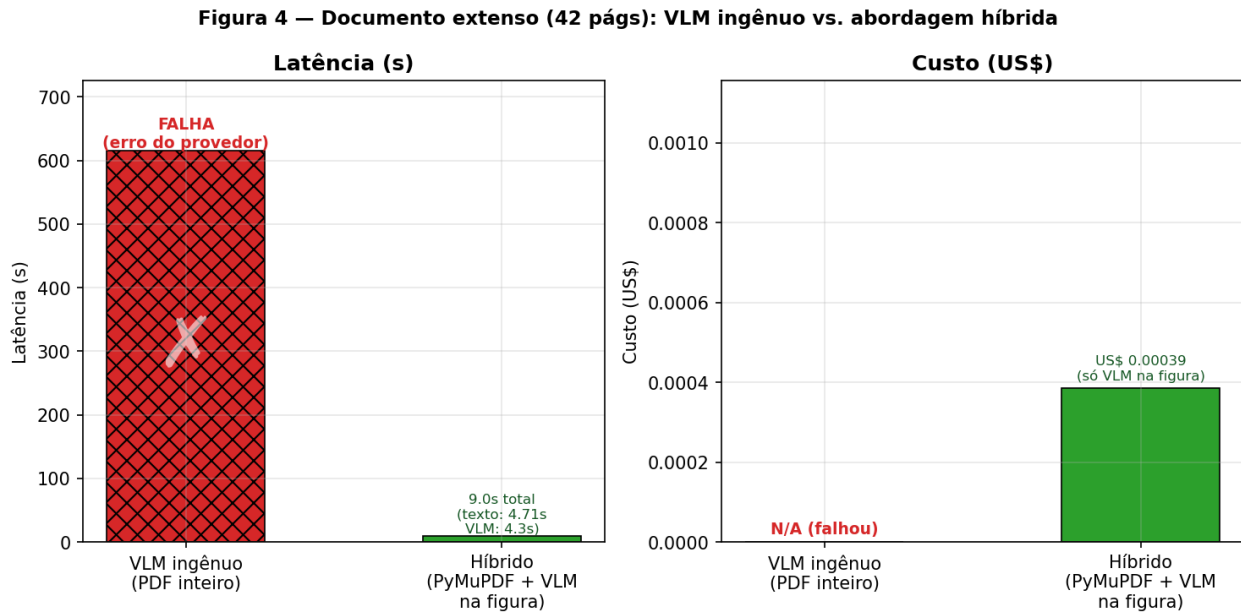


Figure 4: Figura 4 - Abordagem ingênua (VLM no PDF inteiro) vs. híbrida (PyMuPDF + VLM seletivo)

Fonte: elaboração própria. Latência em segundos; custo em US\$ por documento.

Dessa forma, consolida-se a distinção arquitetural fundamental entre os dois casos de uso: documentos de página única ou baixo volume comportam *single-pass* direto ao VLM; documentos extensos exigem pré-processamento determinístico para extração de texto e tabelas, reservando a inferência do VLM para os elementos visuais que genuinamente demandam compreensão multimodal. Essa arquitetura híbrida elimina o risco de falha por *timeout* do provedor, reduz custo de inferência por chamada e mantém a capacidade de leitura de gráficos.

4.4 Estudo de Robustez na CNH: Limites da Escalonabilidade

A CNH (Documento 1. jpeg) constitui o caso mais difícil da matriz, com `status=partial` em 100% dos runs: o campo CPF retorna `None` para todos os modelos e modos, pois o validador determinístico zera valores inválidos. Esse cenário motivou um estudo de robustez estruturado em três experimentos adicionais, cujos achados são enumerados a seguir.

1. *Upscaling* via LANCZOS 3× (de 341×600 px para 1023×1800 px, com ajuste de nitidez 1,5×) não recuperou nenhum campo nos modelos *gemini-2.5-flash-lite* e *gpt-4o-mini*. O placar determinístico

permaneceu em 3/6 para ambos, enquanto o custo do *gpt-4o-mini* aumentou 153% (de US\$0,00222 para US\$0,00563), por conta dos *visual tokens* adicionais gerados pela imagem maior. O resultado é atribuível à natureza da degradação: o *upscaling* interpola pixels existentes sem recuperar informação destruída pela compressão JPEG original; o problema é de **legibilidade intrínseca**, não de densidade de pixels.

2. O prompt ancorado (com descrições de campo vinculadas a posições de layout na CNH, como “campo rotulado ‘DATA EMISSÃO’, diferente da 1ª habilitação e da validade”) recuperou o campo `data_emissao` no modelo *gemini-2.5-flash-lite* sem custo adicional relevante (US\$0,000294 vs. US\$0,000279 no prompt genérico), elevando o placar de 3/6 para 4/6. O mesmo resultado 4/6 foi alcançado pelo *gemini-2.5-pro* (modelo forte) em prompt genérico, demonstrando que a ancoragem de *prompt* iguala o modelo pequeno ao modelo forte neste campo específico.
3. A escalação para o *gemini-2.5-pro* — modelo aproximadamente 57× mais caro por chamada (US\$0,016 vs. US\$0,000286) — não recuperou campos além do que o modelo pequeno com prompt ancorado já atingia. O placar permaneceu em 4/6 em ambas as condições do modelo forte, com CPF e `data_nascimento` irrecuperáveis em qualquer configuração testada. A Tabela 2×2 de ablação é apresentada abaixo para referência dos dados brutos da ablação.

O CPF permanece irrecuperável em todos os quatro runs da ablação (com e sem ancoragem, modelo pequeno e forte): os modelos extraem consistentemente o dígito inicial como 8 em vez de 0, erro atribuído à degradação visual do JPEG original. Nenhum lever de *prompt engineering* ou escalação de modelo corrige um problema de legibilidade que precede a inferência. Assim, estabelece-se a distinção operacional entre **complexidade** (resolúvel por modelo maior ou prompt mais rico) e **legibilidade** (requer pré-processamento de imagem, recorte de região de interesse ou re-captura do documento).

4.5 Confiabilidade da Avaliação: O Juiz Não-Calibrado

O diagnóstico de acurácia na CNH revelou uma limitação metodológica relevante: o *LLM-as-judge* (*gpt-4o*) diverge sistematicamente da métrica determinística em cenários de baixa resolução, com gap de até $\pm 0,17$ (em escala 0–1), ora superestimando ora subestimando a acurácia real. O juiz aprovou campos com datas e nomes incorretos porque a própria imagem de baixa resolução dificulta a verificação visual pelo modelo avaliador; o juiz tende a ser leniente quando não consegue confirmar o valor correto no documento original.

Dessa forma, o relato das duas métricas em paralelo não é redundante: é **indispensável** para honestidade do experimento. A `acuracia_det` fornece âncora objetiva onde há *ground truth* disponível; a `acuracia_juiz` é útil em documentos sem anotação determinística (como a Fatura CELPE), mas deve ser tratada com reserva em condições de baixa resolução. Essa constatação reforça a recomendação de anotação de *ground truth* para documentos-alvo críticos e de validação de campos via regras determinísticas (CPF, CNPJ, linhas digitáveis) como pós-processamento obrigatório, independentemente do modelo de avaliação adotado.

4.6 Fundamentação por Benchmarks de Terceiros

Dado o escopo restrito da POC (n=3 documentos, dois dos quais com *ground truth* disponível apenas parcialmente), a generalização das conclusões é sustentada por benchmarks públicos de grande escala. A Tabela 3 consolida os números relevantes, selecionados para cobrir os casos não testados localmente: gráficos, documentos multidomínio e eficácia do *constrained decoding*.

Tabela 3 - Benchmarks de terceiros utilizados para fundamentação externa dos achados

Benchmark	Tarefa	Métrica	Qwen2.5-VL-7B	Qwen2.5-VL-72B	GPT-4o	MinerU	Ref.
DocVQA	QA sobre documentos	ANLS	95,7	96,4	92,8	—	(Bai et al., 2025)
ChartQA	QA sobre gráficos	<i>relaxed acc.</i>	87,3	89,5	85,7	—	(Bai et al., 2025)
OmniDocBench (NED)	<i>Parsing</i> de PDF	NED (↓ melhor)	—	—	0,144	0,058	(Ma et al., 2024)
OmniDocBench (TEDS)	Tabelas	TEDS	—	—	72,8	79,4	(Ma et al., 2024)
<i>Structured Outputs</i>	Conformidade ao <i>schema</i>	% válido	—	—	~100%*	—	(OpenAI, 2024)

Fonte: elaboração própria com base nas referências indicadas. *Structured Outputs* da OpenAI com XGrammar reporta 100% de conformidade ao *schema* JSON; sem constrição, conformidade cai para menos de 40%. *overhead* do XGrammar menor que 40 μ s/token.

Nessa perspectiva, os dados externos confirmam três achados que a POC não pôde testar diretamente. Primeiro, o modelo *Qwen2.5-VL-7B* (versão pequena, auto-hospedável) atinge desempenho virtualmente idêntico ao *72B* em DocVQA (95,7 vs. 96,4, gap inferior a 1 ponto percentual) e ChartQA (87,3 vs. 89,5), sustentando a viabilidade de modelos menores para documentos estruturados. Segundo, ferramentas de *pipeline* especializadas como MinerU superam o *GPT-4o* em *parsing* de PDF em NED (0,058 vs. 0,144) e em TEDS de tabelas (79,4 vs. 72,8) no OmniDocBench (Ma et al., 2024), corroborando a superioridade da abordagem híbrida para documentos textuais densos. Terceiro, o *constrained decoding* via *Structured Outputs* elimina praticamente a não-conformidade ao *schema* JSON — problema que motivou originalmente a segunda etapa do *pipeline* atual — a custo computacional desprezível (inferior a 40 μ s/token com XGrammar), o que remove o argumento técnico central que justificava a arquitetura de duas inferências (OpenAI, 2024).

Além disso, os serviços de Document AI gerenciados (Azure Document Intelligence, Google Document AI Layout Parser) operam na faixa de US\$1,50 por 1.000 páginas para OCR puro e US\$10 por 1.000 páginas para análise de *layout* com saída em Markdown estruturado; o Google Layout Parser complementa a análise com descrição de figuras via Gemini. Esses valores oferecem ponto de comparação para a análise de viabilidade de custo em escala, apresentada na Seção 6.

Nessa perspectiva, os experimentos empíricos da POC e os benchmarks externos convergem para um conjunto coerente de conclusões: a arquitetura *single-pass* com *structured output* é superior em custo e latência sem sacrifício de acurácia, o documento extenso requer abordagem híbrida, a escolha do modelo certo na classe certa supera a escalação indiscriminada, e o *constrained decoding* elimina a justificativa técnica remanescente para o *pipeline* em duas etapas. A seção seguinte sintetiza essas conclusões em recomendações acionáveis e analisa as implicações para a integração com a plataforma Tech4.ai.

5. Conclusão e Recomendação

A investigação conduzida nas seções anteriores permite formular uma recomendação técnica fundamentada, ancorada tanto nos resultados empíricos da POC quanto nos *benchmarks* de terceiros que cobrem o espectro

mais amplo de documentos e modelos. O presente estudo conclui que a substituição do *pipeline* de duas inferências por um único *Vision-Language Model* (VLM) de porte pequeno, operado em *single-pass* com *structured output* via *constrained decoding*, constitui a rota de modernização mais favorável sob os critérios de latência, custo, acurácia e esforço de integração.

Motor padrão recomendado — VLM proprietário pequeno em *single-pass* com *structured output*. Dentre os motores avaliados, o *Gemini Flash-Lite* opera como referência da classe: colapsa as duas inferências em uma única chamada, garantindo *JSON* conforme ao *schema* por meio de *constrained decoding* (conformidade de 100% reportada pela OpenAI Structured Outputs; *overhead* de XGrammar inferior a 40 μ s/token, negligível na latência total), e lê nativamente tabelas e gráficos sem pré-processamento especial. Os números da POC sustentam essa escolha de forma direta: na fatura CELPE, o modo *single* entregou 3,34 s e US\$ 0,00040 contra 4,90 s e US\$ 0,00059 do *two-step* com o mesmo modelo — redução de aproximadamente 32% em ambas as dimensões, com acurácia do juiz preservada (0,857 vs. 0,833). Na CNH, o *single-pass* foi novamente superior (2,61 s/US\$ 0,00027 vs. 4,14 s/US\$ 0,00038). Em nenhuma célula da matriz o modo de duas etapas superou o *single-pass* em custo ou latência. Adicionalmente, o *gpt-4o-mini* — alternativa aparentemente comparável — custou de 10 a 18 vezes mais que o *Gemini Flash-Lite* (fatura/*single*: US\$ 0,00734 vs. US\$ 0,00040) sem ganho de acurácia detectável, evidenciando que a escolha do modelo pequeno *correto* é tão decisiva quanto a escolha da arquitetura.

Estratégia escalonada com nuance crítica. A recomendação de escalabilidade não é uniforme. Observa-se que escalar para um modelo maior só compensa quando o fator limitante é a **complexidade** do layout ou o raciocínio sobre estruturas visuais ambíguas — e não quando o fator é a **legibilidade** da imagem de entrada. O estudo de robustez conduzido com a CNH de baixa resolução (341×600 px) torna essa distinção concreta: o *upscaling* por Lanczos 3× não recuperou nenhum campo adicional (aumento de 153% no custo do *gpt-4o-mini*, ganho zero); o uso do *Gemini 2.5 Pro* — aproximadamente 57 vezes mais caro que o *Flash-Lite* — também não recuperou campos além do que o *prompt* ancorado já havia obtido gratuitamente; e o CPF permaneceu irrecuperável por limitação intrínseca da imagem, não do modelo. Nesse cenário, o alavancador *correto* é o **pré-processamento** — recorte (*crop*), binarização, ou, idealmente, re-captura da imagem com resolução adequada — não a escala do VLM. Essa nuance é operacionalmente relevante: confundir-la implicaria em aumentar custos sem ganho de qualidade. A estratégia escalonada recomendada é, portanto: (1) *Gemini Flash-Lite* como motor padrão; (2) *Gemini Flash* para documentos com layout complexo ou múltiplas colunas; (3) *prompt* ancorado com âncoras de campo como primeira intervenção diante de campos ausentes; (4) pré-processamento de imagem como segunda intervenção antes de qualquer escalada de modelo.

Documento extenso — abordagem híbrida. Para documentos de múltiplas páginas com conteúdo predominantemente textual, o caminho ingênuo de enviar o arquivo integral ao VLM demonstrou-se inviável: o *paper* de 42 páginas (28 MB em PDF) resultou em falha após aproximadamente 615 s de processamento. A abordagem híbrida resolve o problema de forma pragmática: extração determinística por *parser* (PyMuPDF) para texto e tabelas — 42 páginas em aproximadamente 4,7 s a custo zero — e chamada de VLM seletiva apenas sobre as figuras que demandam interpretação visual ($\approx$ 4,3 s/US\$ 0,00039 por figura). O custo total da rota híbrida para o documento foi de US\$ 0,00039, contra falha total do caminho ingênuo. Essa separação responde a um princípio geral: usar o VLM como *especialista de visão*, não como substituto de *parsers* determinísticos onde estes são mais rápidos, mais baratos e igualmente precisos.

Rota de custo em escala. Para operações em alto volume, o *Gemini Flash-Lite* via API mantém custo unitário baixo, mas incorre em custo marginal por documento. Quando o volume justificar o investimento em infraestrutura de GPU, o *Qwen2.5-VL-7B* — modelo *open-source* de 7 bilhões de parâmetros — apresenta desempenho de 95,7 pontos em DocVQA contra 96,4 do 72B (gap inferior a 1 ponto percentual) e 87,3 em ChartQA, tornando-o candidato viável para *self-hosting*. Nessa configuração, o custo marginal por documento converge a zero, compensando o investimento de infraestrutura a partir de determinado patamar

de volume. A seção seguinte quantifica esse ponto de equilíbrio em termos de GPU-hora.

Equilíbrio entre inovação e viabilidade. Cabe reconhecer as limitações do presente estudo: a POC avaliou apenas três documentos ($n = 3$), e os resultados empíricos têm caráter ilustrativo; a generalização apoia-se nos *benchmarks* de terceiros citados. Não obstante, os achados são internamente coerentes e consistentes com a literatura especializada. A recomendação não persegue o modelo mais recente disponível, mas a **combinação mais viável** entre ganho técnico mensurável e esforço de integração razoável para o contexto da Tech4.ai: preservar o envelope `{status, extracted_data}`, reutilizar o `layout_id` como fonte do *JSON Schema*, manter os *validators* determinísticos (CPF, CNPJ, Linha Digitável) como pós-processamento, e trocar apenas o motor interno de dois LLMs para um único VLM. Dessa forma, a mudança arquitetural proposta é cirúrgica — reduz latência e custo, amplia a capacidade de leitura visual, e não exige redesenho da interface nem da experiência do operador.

6. Análise de Viabilidade de Integração

A viabilidade de substituição do motor de extração atual não se restringe à acurácia dos modelos; envolve, igualmente, custo operacional em escala, requisitos de infraestrutura, compatibilidade arquitetural com o produto existente e conformidade legal quanto à residência de dados sensíveis. Esta seção examina cada uma dessas dimensões para as três rotas da *shortlist* — A (VLM proprietário pequeno), B (VLM *open-source self-hosted*) e C (Document AI gerenciado + LLM pequeno) — e encerra com considerações sobre proteção de dados à luz da LGPD.

6.1 Custo Operacional em Escala

Os custos por documento mensurados na POC (seção 4) permitem extrapolar estimativas por mil documentos para cada rota via API. A Tabela 4 consolida esses valores, incluindo a opção de *self-host* da Rota B.

Tabela 4 - Estimativa de custo por 1.000 documentos: API por modelo vs. *self-host*

Rota	Motor	Custo/documento (medido)	Custo/1.000 docs (API)	<i>Self-host</i> GPU	Custo/1.000 docs (<i>self-host</i>)	Volume de equilíbrio ¹
A	Gemini 2.5 Flash-Lite (<i>single-pass</i>)	US\$ 0,00027–0,00040	US\$ 0,27–0,40	—	—	—
A	GPT-4o-mini (<i>single-pass</i>)	US\$ 0,00226–0,00734	US\$ 2,26–7,34	—	—	—
A (API)	Qwen2.5-VL-72B (via Open-Router)	US\$ 0,00051–0,00131	US\$ 0,51–1,31	—	—	—
B	Qwen2.5-VL-7B (<i>self-hosted</i>)	custo marginal ≈ 0	≈ 0 (marginal)	A10G 24 GB VRAM, $\sim$ US\$ 0,75–1,00/h	US\$ 7,50–10,00 ²	$\sim$ 150 k–2,2 M docs/mês ³

Rota	Motor	Custo/documento (medido)	Custo/1.000 docs (API)	Self-host GPU	Custo/1.000 docs (self-host)	Volume de equilíbrio ¹
C	Azure/Google Layout + LLM pequeno	—	US\$ 10,30– 11,00 ⁴	—	—	—

Fonte: elaboração própria com base nos resultados da POC (benchmark/results/results.json) e nas tabelas de preço dos provedores.

¹ Volume mensal a partir do qual o custo acumulado do *self-host* torna-se inferior ao custo acumulado da rota API de referência, assumindo GPU dedicada rodando em regime contínuo (24 h/dia, 30 dias = ~US\$ 720/mês). ² Assumindo throughput de 100 documentos/hora na GPU A10G (estimativa conservadora para inferência de um VLM-7B em FP16/INT4); instâncias equivalentes disponíveis em provedores como Lambda Labs, RunPod e AWS EC2 (g5.xlarge). ³ Ponto de equilíbrio vs. Gemini Flash-Lite (US\$ 0,00033/doc médio): $720/0,00033 \approx 2,18$ M docs/mês. Vs. GPT-4o-mini (US\$ 0,0048/doc médio): $720/0,0048 \approx 150$ k docs/mês. ⁴ Azure AI Document Intelligence Layout a US\$ 10,00/1.000 páginas (Google Document AI Layout Parser equivalente) (Microsoft, 2024; Google, 2024) + LLM pequeno (Gemini Flash-Lite) estimado em US\$ 0,30–1,00/1.000 docs.

Dessa forma, a Rota A com Gemini 2.5 Flash-Lite é, por ampla margem, a mais barata em qualquer volume — US\$ 0,27–0,40/1.000 documentos — ao passo que o GPT-4o-mini encarece a mesma rota em 10–18 vezes sem ganho de acurácia correspondente, conforme verificado na matriz da POC (seção 4). A Rota B (*self-host*) só compensa economicamente a partir de volumes da ordem de 150 mil a 2,2 milhões de documentos por mês, dependendo do modelo API de referência, e exige que a GPU opere em regime de alta ocupação; abaixo desse patamar, o custo fixo de infraestrutura supera o custo variável de API. A Rota C, embora conservadora em termos de risco técnico, apresenta o maior custo operacional entre as rotas analisadas.

6.2 Requisitos de Infraestrutura

Cada rota impõe exigências distintas de implantação, manutenção e maturidade operacional:

1. **Rota A — VLM proprietário pequeno via API:** nenhuma infraestrutura adicional além de uma chave de API e conectividade HTTPS. O gerenciamento de modelos, escalabilidade, redundância e atualizações de versão são responsabilidade do provedor. É a rota de menor *time-to-market* e de mais baixo risco operacional; adequada para estágios iniciais e para equipes sem *expertise* em MLOps.
2. **Rota B — VLM open-source self-hosted (Qwen2.5-VL-7B):** exige GPU com no mínimo 16–24 GB de VRAM (ex.: NVIDIA A10G ou RTX 4090) para inferência em FP16; em INT4 (*quantized*), é possível reduzir para 8 GB de VRAM com degradação mínima de acurácia. Além do hardware, requer: (a) servidor de inferência (*vLLM*, *TGI* ou equivalente); (b) fila de tarefas assíncronas para absorver picos de volume; (c) *pipeline* de MLOps para monitoramento de *drift*, atualização de versão e *rollback*; (d) time com competência em infraestrutura de ML. O custo de capital humano e operacional deve ser somado ao custo de GPU ao avaliar o TCO (*Total Cost of Ownership*).
3. **Rota C — Document AI gerenciado + LLM pequeno:** dois serviços distintos orquestrados em sequência — (a) chamada ao serviço de OCR/layout (Azure AI Document Intelligence Layout ou Google

Document AI Layout Parser) para converter o documento em Markdown estruturado; (b) chamada ao LLM pequeno para extração tipada a partir do Markdown. Implica duas integrações de API, dois contratos de SLA, dois pontos de falha e latência composta (2–4 s/página de OCR + latência do LLM). A vantagem é que ambos os serviços são totalmente gerenciados, sem GPU própria, mas o *pipeline* é mais complexo que a Rota A.

6.3 Integração com a Plataforma Tech4.ai

Nessa perspectiva, o aspecto de maior valor estratégico da abordagem proposta é que a substituição do motor de extração pode ser realizada de forma *drop-in*, sem alterar nenhum elemento visível ao cliente. A análise da documentação oficial da Tech4.ai (TECH4.AI, 2024) evidencia quatro pontos de compatibilidade direta:

1. **Preservação do envelope de resposta:** o contrato `{"status": "ok"|"partial"|"error", "extracted_data": {...}}` permanece inalterado. Um VLM com *structured output (constrained decoding)* emite o objeto JSON diretamente na única chamada de inferência, derivando "partial" quando campos opcionais retornam null após validação — exatamente a semântica atual.
2. **Reuso do layout_id como fonte do JSON Schema:** cada layout já define, por campo, nome, tipo de dado e descrição em linguagem natural. Esses três atributos mapeiam 1:1 para as propriedades de um JSON Schema ("type", "description") que pode ser passado ao parâmetro `response_format.json_schema` do VLM. Dessa forma, o *visual builder* existente e o fluxo de configuração de layouts pelo cliente permanecem intactos; apenas o motor de inferência interno é trocado.
3. **Manutenção dos validators determinísticos como pós-processamento:** os *validators* nativos de CPF, CNPJ, UF e Linha Digitável de Boletão são algoritmos determinísticos independentes do modelo de IA; sua lógica de retornar null para valores inválidos não tem relação com a etapa de inferência. Eles devem continuar rodando **após** a saída do VLM, como camada de pós-processamento, garantindo paridade semântica plena com o comportamento atual e preservando a qualidade dos dados extraídos.
4. **Compatibilidade de transporte:** o endpoint POST `https://api.tech4.ai/document/extract/`, o *header* de autenticação, os parâmetros `file_url` e `layout_id`, e os códigos HTTP (400, 401, 404) permanecem inalterados. A mudança de 2 LLMs para 1 VLM *single-pass* é totalmente interna, invisível para o cliente que consome a API.

Assim, o ganho operacional — eliminação de uma chamada de inferência, redução de latência de $\approx 32\%$ e redução de custo proporcional (seção 4) — é obtido sem nenhuma quebra de contrato e sem impacto para as integrações já existentes dos clientes da plataforma.

6.4 LGPD e Residência de Dados

Os documentos processados pela plataforma Tech4.ai incluem dados pessoais sensíveis no sentido da Lei Geral de Proteção de Dados (Brasil, 2018): a CNH contém nome completo, CPF, data de nascimento e número de registro; a fatura de energia contém nome, endereço e CPF do titular. O processamento desses dados por modelos via API de terceiros implica que o dado transita e é processado fora da infraestrutura do controlador, o que exige base legal adequada, cláusulas contratuais de processamento de dados com o provedor e avaliação da localização dos servidores.

Nesse contexto, a Rota B (*self-host*) oferece a maior proteção de residência de dados: o documento nunca sai da infraestrutura controlada pelo operador, eliminando o risco de transmissão transfronteiriça. Essa característica é relevante especialmente para clientes da Tech4.ai em setores regulados (financeiro, saúde, setor público) onde exigências contratuais ou normativas impõem processamento *on-premise* ou em nuvem soberana. A Rota A, por sua vez, depende das políticas de privacidade de cada provedor de API (Google, OpenAI); ambos afirmam não treinar modelos com dados de clientes via API em suas camadas pagas, mas o dado egresso permanece como superfície de risco a ser avaliada pelo encarregado de proteção de dados (DPO) da organização. A Rota C apresenta o mesmo perfil da Rota A em termos de residência, acrescido do risco de dois provedores distintos receberem o dado.

Além disso, deve-se considerar que serviços de Document AI gerenciados (Azure, Google) oferecem opções de implantação em regiões específicas (incluindo regiões no Brasil), o que pode mitigar parte do risco de transferência internacional, ainda que não elimine a exposição ao processamento por infraestrutura de terceiros.

Por fim, a análise de viabilidade converge para a Rota A como ponto de entrada recomendado — menor custo, zero infraestrutura adicional, integração *drop-in* e *time-to-market* imediato —, com a Rota B reservada para cenários de alto volume ou requisitos estritos de residência de dados, conforme discutido na seção 5.

Referências

ANTHROPIC. **Structured Outputs**. San Francisco: Anthropic, 2025. Disponível em: <https://platform.claude.com/docs/en/basics/with-claude/structured-outputs>. Acesso em: jun. 2026.

AWS. **Amazon Textract Pricing**. Seattle: Amazon Web Services, 2025. Disponível em: <https://aws.amazon.com/textract/pricing>. Acesso em: jun. 2026.

BAI, Shuai et al. **Qwen2.5-VL Technical Report**. [S.l.]: Alibaba Group, 2025. Disponível em: <https://arxiv.org/abs/2502.13923>. Acesso em: jun. 2026.

BORCHMANN, Łukasz; PIETRUSZKA, Michał; STANISLAWEK, Tomasz; JULKA, Dawid; GRZEGORZEK, Karol. **Towards a Multi-Task Learning Setup for Document Information Extraction**. In: *Proceedings of the EMNLP Workshop*, 2021. Disponível em: <https://aclanthology.org/2021.emnlp-main.670>. Acesso em: jun. 2026.

BRASIL. **Lei n. 13.709, de 14 de agosto de 2018**: Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/113709.htm. Acesso em: jun. 2026.

DONG, Yixin et al. **XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models**. [S.l.]: MLSys, 2025. Disponível em: <https://arxiv.org/abs/2411.15100>. Acesso em: jun. 2026.

DOTTXT. **Coalescence: Making Structured Generation Fast**. [S.l.]: dottxt, 2024. Disponível em: <https://blog.dottxt.ai/coalescence.html>. Acesso em: jun. 2026.

DOTTXT. **Say What You Mean: Constrained Generation and Its Discontents**. [S.l.]: dottxt, 2024. Disponível em: <https://blog.dottxt.ai/say-what-you-mean.html>. Acesso em: jun. 2026.

GOOGLE. **Document AI Pricing**. Mountain View: Google, 2024. Disponível em: <https://cloud.google.com/document-ai/pricing>. Acesso em: jun. 2026.

GOOGLE. **Gemini API Structured Output**. Mountain View: Google, 2025. Disponível em: <https://ai.google.dev/gemini-api/docs/structured-output>. Acesso em: jun. 2026.

GOOGLE. **Layout Parser**. Mountain View: Google, 2024. Disponível em: <https://docs.cloud.google.com/document-ai/docs/layout-parse-chunk>. Acesso em: jun. 2026.

HUANG, Yupan et al. **LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking**. In: ACM INTERNATIONAL CONFERENCE ON MULTIMEDIA, 30., 2022. Disponível em: <https://arxiv.org/abs/2204.08387>. Acesso em: jun. 2026.

KIM, Geewook et al. **OCR-Free Document Understanding Transformer (Donut)**. In: EUROPEAN CONFERENCE ON COMPUTER VISION (ECCV), 2022. Disponível em: <https://arxiv.org/abs/2111.15664>. Acesso em: jun. 2026.

MA, Yahui et al. **OmniDocBench: Benchmarking Document Parsing with Diverse Scalable Data**. In: IEEE/CVF CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION (CVPR), 2025, Seattle. *Proceedings...* Seattle: IEEE, 2025. Disponível em: <https://arxiv.org/abs/2412.07626>. Acesso em: jun. 2026.

MASRY, Ahmed et al. **ChartQA: A Benchmark for Question Answering about Charts with Visual and Logical Reasoning**. In: FINDINGS OF THE ACL, 2022. p. 2263–2279. Disponível em: <https://aclanthology.org/2022.findings-acl.177>. Acesso em: jun. 2026.

MATHEW, Minesh; KARATZAS, Dimosthenis; JAWAHAR, C. V. **DocVQA: A Dataset for VQA on Document Images**. In: IEEE WINTER CONFERENCE ON APPLICATIONS OF COMPUTER VISION (WACV), 2021. Disponível em: <https://arxiv.org/abs/2007.00398>. Acesso em: jun. 2026.

MICROSOFT. **Azure AI Document Intelligence Pricing**. Redmond: Microsoft, 2024. Disponível em: <https://azure.microsoft.com/en-us/pricing/details/document-intelligence/>. Acesso em: jun. 2026.

OPENAI. **API Pricing**. San Francisco: OpenAI, 2025. Disponível em: <https://platform.openai.com/docs/pricing>. Acesso em: jun. 2026.

OPENAI. **Introducing Structured Outputs in the API**. San Francisco: OpenAI, 2024. Disponível em: <https://openai.com/index/introducing-structured-outputs-in-the-api/>. Acesso em: jun. 2026.

OPENAI. **Structured Outputs (Guia)**. San Francisco: OpenAI, 2025. Disponível em: <https://platform.openai.com/docs/guides/structured-outputs>. Acesso em: jun. 2026.

OPENROUTER. **Structured Outputs**. [S.l.]: OpenRouter, 2026. Disponível em: <https://openrouter.ai/docs/guides/features/structured-outputs>. Acesso em: jun. 2026.

OPENROUTER. **Usage Accounting**. [S.l.]: OpenRouter, 2026. Disponível em: <https://openrouter.ai/docs/cookbook/administration/usage-accounting>. Acesso em: jun. 2026.

QWEN. **Qwen2.5-VL**. [S.l.]: Alibaba Cloud, 2025. Disponível em: <https://qwen.ai/blog?id=qwen2.5-vl>. Acesso em: jun. 2026.

TAM, Zhi Rui et al. **Let Me Speak Freely? A Study of the Impact of Format Restrictions on LLM Performance**. In: PROCEEDINGS OF EMNLP 2024 INDUSTRY TRACK. [S.l.]: ACL, 2024. Disponível em: <https://aclanthology.org/2024.emnlp-industry.91/>. Acesso em: jun. 2026.

TECH4.AI. **Vision: Doc Extraction API — Documentação oficial**. 2024. Disponível em: <https://docs.tech4.ai/vision/doc-extraction-api>. Acesso em: jun. 2026.

TECH4.AI. **Vision: Layout Configuration**. 2024. Disponível em: <https://docs.tech4.ai/vision/layout-config>. Acesso em: jun. 2026.